UNIT – 4 (Part-1)
Text Editing With vi and nano Editor, Shell Programming

Topic: 1 Explain VI editor in detail Mark 5

Ans.

Detailed:

Introduction to VI editor

- No matter what work you do with the UNIX system, you will eventually write some C programs or shell scripts. You may have to edit some of the system files at times.
- For all this you must learn to use an editor, and UNIX providers a very versatile one VI editor, VI editor is a screen editor, where a portion of the file is displayed on the terminal screen, and the cursor can be moved around the screen to indicate where you want to make changes.
- You can select which part of the file you want to have displayed.
- Screen editors are also called display editors, or visual editors. VI is one of the more popular screen editors that run on the UNIX system.

Invoking VI

- It will put filename into a buffer and display the file on the screen.
- If the file is larger than the screen can display, the screen will act as a window into the file.
- At the beginning of a session, the screen will display the first part of the file.

If filename does not exist, VI will create it.

Modes in VI editor

- One of the most important aspects to remember about **vi** is that most of the commands fall into one of three modes:

(1) Command mode:

- This is the default mode of VI editor. This mode is used to give some command for navigation, edition and copy or cut.
- This is the base mode of VI editor if user want to switch this mode to **insert or input mode then user has to press i or I or a or A.**
- If user wants to switch execute mode from this mode then user has to **press:**

(2) Input or Insert Mode:

- This mode is used to enter data in VI editor. If user wants to insert some text to the editor then first user has to select this mode.
- **This mode will be selected from command mode pressing I or i or a or A.**

(3) Execute Mode:

- This mode is used to save or quit from VI editor. Whatever the changes user has done in file using VI editor if user wants to save, find any particular string then using this mode user can save or find the string and also whenever user wants to quit from the VI editor this mode will supports user to quit from the VI editor.
- **To switch to this mode user has to press: or / on command mode**

GRACE COLLEGE

- **Switching mode in VI**

- While you start **VI** editor at that time you will be at vi mode that will ready to accept defined command on that particular key but not any input.
- If you want to input text into the file you will have to go to input mode for that you will have to **press "i".**
- If you will press **"i" at VI** mode you will be at input mode here you can input any data into the file from this mode if you want to switch to vi mode then you will have to press **"Esc"** key.
- If you will press **"Esc"** key at input mode you will be able to switch to vi mode.
- If you want to switch to command mode from the VI mode then you will have to press **":" or "/"** that will support you to switch you from VI mode to command mode.
- If you want to switch to command mode from the input mode then you will have to first come to the VI mode then you will be able to switch to command mode.
- To switch from input mode to command mode then you first will have to press **"Esc"** key and then after you will have to press **":" or "/"** key that will support you to switch from input mode to command mode via **VI** mode.

Cursor movement in VI editor

- Whatever the command you are giving that will work on VI mode only.
- To move around in the file, use the arrow keys: the up arrow will move the cursor up one line, the down arrow down one line.
- The right arrow will cause the cursor to move one position to the right.
- From the end of a line, it will go to the beginning of the next line.
- For terminals with no functional arrow keys, four keys will move the cursor around:

h	**left arrow**
j	**down arrow**
k	**up arrow**
l	**right arrow**

Screen control commands

- Whatever the command you are giving that will work on **vi** mode only.

ctrl + f	This will move the user forward one screen in the file.
ctrl+b	This will move the user backward one screen in the file.
G	This will move the cursor to the end of the file. Note, again that vi, like any other UNIX utility, is always case sensitive.
Num	This will bring the cursor to line defined line number.

Entering, Editing, Copying data into vi editor

- Whatever the command you are giving that will work on vi mode only.

x	That will delete on character.
dd	That will delete lines beginning at the current line.
yy	That will copy lines beginning at the current line.
p	That will paste copied line.
u	This is very useful command. It cancels the effect of the previously executed command.

GRACE COLLEGE

Q	When in vi, typing will bring the user into command mode. As the : is typed, it will be displayed n the last line of the screen, and vi will wait for a command to be typed. q is such a command. This will exit vi, if no changes have been made since the last write-to-file command :q! will exit, even if the buffer has not been written to the file.
w	Will cause the contents of the buffer to be written to the file :wq will write the buffer to the file, and exit the calling shell.
I	This will support you to switch from VI mode to input mode.
:	This will support you to switch from VI mode to command mode.
esc	This will support you to switch from input mode to VI mode.
/	This will support you to find any particular pattern from the file.

Introduction of nano editor

- nano is a simple text editor
- It was created as part of the GNU project started by Richard Stallman
- Most Linux distributions contain the vi and/or vim editors
- These editors are often used by System Administrators
- And you can always download the emacs editor
- emacs is preferred by programmers
- But both of these editors take some time to learn
- nano does not have the full set of features of these editors
- But it is great for people who are just starting to work with Unix
- Or who want to create a file quickly with a minimum of fuss

Running nano

- To run nano you type nano followed by the name of the file

nano FILENAME

- If the file does not yet exist it will be created for you
- So if I wanted to create a file named .bash_profile I would type

nano .forward

- This will bring up a text window with the name of the file on the top line

nano Commands

- All command in nano are run with the help of the Control key
- Control key is labeled Cntrl on all machines except Macs
- In the list of commands the symbol ^ stands for the Control key
- To quit nano hold down the Control key and hit X
- The save command in nano is ^O WriteOut
- You might expect it to be ^S Save

- Unfortunately Control S has special meaning in Unix
- So it cannot be used for this purpose
- The same problem exists for Control Q
- So you have to use Control X to quit

Editing Text in nano

- To enter text in a file, simply type it in the window
- You can use the arrow keys to move around in the text window
- And the Backspace or Delete keys to delete a character
- You can move to the end of a line with Control E
- And the beginning of the line with Control A
- nano has a very simple cut and paste feature
- But it only works on the whole line of text
- You use the arrow keys to get to a line you want to move
- And cut it with Control K
- You can then move to another position in the file
- And use Control U to paste the line into this new position

Save and Quit

- To save the file, hold down the Control key while pressing O
- When you do this, a line will appear at the bottom of the screen
  File Name to Write:

- This line is asking you to confirm the name of your file
- Hit Enter or Return to accept this filename
- To quit, hold down the Control key while pressing X

Searching in nano

- nano has a very basic search feature
- If you hit Control W (for Where) a line will appear at the bottom of the screen Search:
- Type in your search text and hit Enter or Return
- The text entry point will move to the first line that has the text
- The search is not case sensitive

UNIT – 4 (Part-2)

Topic: 1 What are Shell keywords? Mark 1

Ans.

Detailed:

- Keyword is a word which is having specific meaning for particular language.
- In programming different words are used like, if, for, while, until, case etc. these words are called shell keywords.

GRACE COLLEGE

Topic: 2 Explain Shell Variables. Mark 2

Ans. Detailed:

- Variable is the location in the memory which is used to store the data.
- A variable is a character string to which we assign a value.
- The value assigned could be a number, text, filename, device, or any other type of data.
- The name of a variable can contain only letters (a to z or A to Z), numbers (0 to 9) or the underscore character (_).
- The following examples are valid variable names −

_RAM
TOKEN_A
VAR_1
VAR_2

- Following are the examples of invalid variable names –

2_VAR
-VARIABLE
VAR1-VAR2
VAR_A!

→ **Defining Variables**

Variables are defined as follows −
variable_name=variable_value

- Shell enables you to store any value you want in a variable. For example −

VAR1="Rohit Sharma" VAR2=100

→ **Accessing Values**

- To access the value stored in a variable, prefix its name with the dollar sign (**$**)

```
#!/bin/sh
NAME="Rohit Sharma"
echo $NAME
```

- The above script will produce the following value −

Rohit Sharma

Topic: 3 Explain types of Variables in Linux. Mark 5

Ans.

Detailed:

- There are three types of variables.

1. **Shell Variables**
2. **System Variables or Environment Variable**
3. **Users define variable**

1. **Shell Variables**

- These variables are used to configure the shell.
- The shell provides string valued variables.

GRACE COLLEGE

Example:

- User=root, which assigns values to the variable user.
- The value of variable is substituted by preceding name with $.

- Other Shell parameters:

$* : It is the set of positional parameters as a single string.
$@: It is same as $* except when enclosed in double quotes.
$? : The exit status of the last command executed as a decimal string.
$# : The number of positional parameters.
$$: The process number of this shell (in decimal).
$! : The process number of the last process runs in the background.

2. System Variables

- These variables are used to configure the environment during the boot sequence or after user login, so they are also known as environment variables.
- The shell has two prompts stored in PS1 and PS2.

PS1: It is the primary prompt string like $ or %. The shell uses the primary prompt when the command is expected by the shell. For changing the value of this prompt, we can use **PS1= '@'.**
As soon as it is set the new prompt is displayed. It can be restored back to $ prompt as PS1="$".

PS2: It is the secondary prompt; it is used when the command entered on the first line was not able to be completed. The default prompt is >.

PATH: Searches the path for commands. If $PATH is not set then the current directory, /bin and /usr/bin are searched by default. Otherwise $PATH consists of directory names separated by:
For Example: PATH=:/usr/Bhavik/bin:/bin:/usr/bin

HOME: It is path name for the home directory. When the user logs in the system, unix normally places the user in home directory or login directory and is the path stored in variable HOME.
The change the default argument for the cd command.
For Example: **cd /usr/Bhavik/bin**

LOGNAME: It contains login name which is stored in the passwd file. This variable shows your username only while Who am I also displays the user name with the terminal type and login name.

MAIL: It displays the absolute path name of user's mailbox and determines where all incoming mail addressed to the user is to be stored. This variable is typically set in the file .profile, in the user's login directory.
For Example: MAIL=/var/spool/mail/root

IFS: This variable contains internal field separators. These are the characters that separate the parameters and commands. These separators include space, tab and newline. Default separator is space.
SHELL: This variable contains the pathname of login shell. Various shells available in unix system include korn shell, bash shell or c shell. User can select any one from them.
TERM: It indicates the Terminal type being used. The value of this variable can be utilised by VI command. The VI editor makes the use of the control file in the directory /usr/lib/terminfo.

MAILCHECK: It contains Interval between tests for new email. The default is 600 seconds. When the user logs in, shell checks mail file to determine if there is new mail. If mail is there, it display "You have a mail".

GRACE COLLEGE

3. **User defined variables**

- The variable which is created by the user is known as user defined variables.

1. The variable name should not be the same as one of the predefined variables.
2. The name of the variable must start with an alphabet or underscore.
3. Unix is case sensitive, so SUM and sum are treated as different variables.

➔ **set, unset, and echo command with variables**

- You can use set command to set any variable.
- You can use unset command to unset/remove any variable.
- You can use echo command to print the value of any variable.

Topic: 4 Explain Positional parameters. Mark 3

Ans.

Detailed:

- A **positional parameter** is a variable within a shell program; its value is set from an argument specified on the command line that invokes the program.
- **Positional parameters** are numbered and are referred to with a preceding ``$'': $1, $2, $3, and so on.
- **Example:**

$sh filename.sh	first	second	third	fourth	fifth	sixth	seventh	eighth	ninth
$0	$1	$2	$3	$4	$5	$6	$7	$8	$9

- Arguments are user supplied data that follow the file name on the command line and are input to the shell script.
- Positional Parameters are predefined memory variables in the shell scripts.
- There are nine positional parameters $1, $2……. $9 that are used to store the user arguments.
- The file name is stores in $0.
- When script is executed the shell puts the first argument in first positional parameter $1, second argument in second positional parameter $2 and so on.

Topic: 5 Explain Decision statements in Linux. Mark 5

Ans.

Detailed:

- While writing a shell script, there may be a situation when you need to adopt one path out of the given two paths. So, you need to make use of **conditional statements** that allow your program to make correct decisions and perform the right actions.
- Unix Shell supports **conditional statements** which are used to perform different actions based on different conditions.
- Unix shell provides decision making using if then else and case structure.

GRACE COLLEGE

If – then – fi

Syntax:

```
If [condition]
then
execute command if condition is true
…
fi
```

Example:

```
NAME="Rohit"
if [ $NAME = "Rohit" ]
then
echo "True - my name is Rohit"if
```

If – else – fi

- If given condition is true, then command1 is executed else command2 is executed.

Syntax:

```
If [condition]
then
        execute command1 if condition is true
else
        execute command2 if condition is false
fi
```

Example:

```
NAME="Virat"
if [ $NAME = "Rohit" ]
then
 echo "True - my name is Rohit"
else
echo "False"
echo "My name is $NAME"
fi
```

if...elif...else...fi

- The if...elif...fi statement is the one level advance form of control statement that allows Shell to make correct decision out of several conditions.

Syntax:

```
if [ condition 1 ]
then
    Statement to be executed if condition 1 is true
    elif [ condition 2 ]
    then
    Statement to be executed if condition 2 is true
    else
    Statement to be executed if no expression is true
    fi
```

GRACE COLLEGE

Example:

```
NAME="Rohit"
if [ $NAME = "Virat" ]
then
echo "Virat Kohli"
elif [ $NAME = "Rohit" ]
then
echo "Rohit Sharma"
else
echo "This is Dhoni"
fi
```

The case...esac Statement

- Unix Shell supports case...esac statement which handles exactly this situation, and it does so more efficiently than repeated if...elif statements.
- There is only one form of case...esac statement

Syntax:

```
case "$variable" in
"$condition1" )
command...
 ;;
 "$condition2" )
command...
;;
esac
```

Example:

```
mycase=1
case $mycase in
1) echo "You selected unix";;
2) echo "You selected php";;
3) echo "You selected java";;
4) echo "You selected c++";;
exit esac
```

Topic: 6 Explain test command in Linux. Mark 2

Ans.

Detailed:

- The test command is used to check file types and compare values. Test is used in conditional execution.

Syntax:

```
test condition
```

OR

```
test condition && true-command
```

OR

```
test condition || false-command
```

GRACE COLLEGE

OR

test condition && true-command || false-command

Example :

- Type the following command at a shell prompt (is 5 greater than 2?):

```
test 5 -gt 2 && echo "Yes"
test 1 -lt 2 && echo "Yes"
```

Sample Output:

Yes Yes

Topic: 7 Explain logical operators in Linux. Mark 3

Ans.

Detailed:

- **Logical Operators**: They are also known as Boolean operators.
- These are used to perform logical operations.
- They are of 3 types:

- **Logical AND (&& / -a)**: This is a binary operator, which returns true if both the operands are true otherwise returns false.
- **Logical OR (|| / -o)**: This is a binary operator, which returns true is either of the operand is true or both the operands are true and returns false if none of them is false.
- **Not Equal to (!)**: This is a unary operator which returns true if the operand is false and returns false if the operand is true.

Example:

```
# take a number from the user
echo "Enter a number: "
read a

# check
if [ `expr $a % 2` == 0 -a $a -gt 10 ]
then
echo "$a is even and greater than 10."
else
echo "$a failed the test."
if
```

Topic: 8 Explain looping structure in Linux. Mark 5

Ans.

Detailed:

- There is total 2 looping statements which can be used in bash programming

1. **while statement**
2. **for statement**

- To alter the flow of loop statements, two commands are used they are,

GRACE COLLEGE

1. **break**
2. **continue**

- Their descriptions and syntax are as follows:

➔ **while statement**

- Here command is evaluated and based on the result loop will executed, if command raise to false then loop will be terminated.

Syntax:

```
while command
do
Statement to be executed
done
```

Example:

```
a=0
# -lt is less than operator

#Iterate the loop until a less than 10
while [ $a -lt 10 ]
do
# Print the values
 echo $a

# increment the value
a=`expr $a + 1`
done
```

➔ **for statement**

- The for loop operate on lists of items.
- It repeats a set of commands for every item in a list.
- Here var is the name of a variable and word1 to wordN are sequences of characters separated by spaces (words).
- Each time the for loop executes, the value of the variable var is set to the next word in the list of words, word1 to wordN.

Syntax:

```
for var in word1 word2 ...wordn
do
Statement to be executed
done
```

Example:

```
#Start of for loop
for a in 1 2 3 4 5 6 7 8 9 10
do
# if a is equal to 5 break the loop
if [ $a == 5 ]
then
break
fi
```

GRACE COLLEGE

```
    # Print the value
    echo "Iteration no $a"
done
```

➔ **until loop**

- As an exit control loop, until loop is available.

Syntax:

```
until [condition]
do
    command1
    command2
done
```

Example:

```
a=10

until [ $a -lt 10 ]
do
echo $a a=`expr $a + 1` done
```

➔ **The break Statement**

- The **break** statement is used to terminate the execution of the entire loop, after completing the execution of all the lines of code up to the break statement.
- It then steps down to the code following the end of the loop.

Syntax:

The following **break** statement is used to come out of a loop −

```
break
```

Example:

```
a=0

while [ $a -lt 10 ]
do
echo $a
if [ $a -eq 5 ]
then
break
fi
a=`expr $a + 1`
done
```

➔ **The continue statement**

- The **continue** statement is like the **break** command, except that it causes the current iteration of the loop to exit, rather than the entire loop.
- This statement is useful when an error has occurred, but you want to try to execute the next iteration of the loop.

GRACE COLLEGE

Syntax:

```
continue
```

Example:

```
NUMS="1 2 3 4 5 6 7"

for NUM in $NUMS
do
Q=`expr $NUM % 2`
if [ $Q -eq 0 ]
then
echo "Number is an even number!!"
continue
fi
echo "Found odd number" done
```

Topic: 9 Explain Arithmetic in shell. Mark 3

Ans.

Detailed:

- Operators are used for manipulating variables and constants in shell programs.
- They are required to perform mathematical operations.

➔ **Shell Arithmetic Operators**

- **Addition: +**
- **Subtraction: –**
- **Multiplication: ***
- **Division: /**
- **Modulus: %**

Each of these operators performs the operation on two integer variables or constants.

For Example, the below program illustrates each of these operations:

```
$ c=`expr $a + $b`
$ echo “the value of addition=$c”
$ d=`expr $a - $b`
$ echo “the value of subtraction=$d”
$ e= expr $a \* $b`
$ echo “the value of multiplication=$e”
$ f=`expr $a / $b`
$ echo “the value of division=$f”
$ g= echo `expr $a % $b`
$ echo “the value of modulus=$c”
```

Topic: 8 Explain Array in Linux.

- An array is a structured arrangement of similar data elements. Within shell scripting, an array is a variable that holds multiple values, whether they are of the same type or different types

GRACE COLLEGE

How to Declare Array in Shell Scripting?

1. Indirect Declaration:

ARRAYNAME[INDEXNR]=value

2. Compound Assignment:

ARRAYNAME=(value1 value2..... valueN)

Alternatively, you can use index numbers to assign values explicitly:

ARRAYNAME=([1]=10 [2]=20 [3]=30)

Example:array.sh

```
#!/bin/bash
#Script to print the keys of the array
#Declaring the Array
declare -a example_array=( "Welcome""To""Geetanjali" )
#Printing the Keys
echo "${!example_array[@]}"
```

Printing the keys of an Array : ${!ARRAY_NAME[index]}

Finding Array Length : ${#ARRAY_NAME[@]}

Adding Elements to an Array : ARRAY_NAME[index_n]="New Element"

Deleting an Element from an Array : unset ARRAY_NAME[index]

Deleting the Entire Array : unset array_Name

Loop through the Array:

```
#!/bin/bash
#Script to print all keys and values using loop through the array
declare -a example_array=( "Welcome""To""Javatpoint" )
#Array Loop
for i in "${!example_array[@]}"
do
echo The key value of element "${example_array[$i]}" is "$i" done
```

Topic: 8 Explain Function in Linux. **Mark - 5**

With the help of functions, overall functionality of a function can be divided into smaller or logical parts, which can be called to perform their task.

- **To create a Function:**

Syntax:

```
function functionName () {
        Commands to be executed
}
```

GRACE COLLEGE

Example:

```
#!/bin/bash
demo () {
        echo 'Welcome to Geetanjali'
       }
       demo
```

- **Passing Arguments**

```
#!/bin/bash
#Script to pass and access arguments
function_arguments()
   {     echo $1
   echo $2
   echo $3
   echo $4
   echo $5     }

#Calling function_arguments
function_arguments "We""welcome""you""on"grace."
```

- **Variable Scope**

Global variables are defined as the variables which can be accessed anywhere within the script regardless of the scope. By default, all the variables are defined as global variables, even if they are declared inside the function. We can also create variables as a local variable. Local variables can be declared within the function body with the ?local? keyword when they are assigned for first time. They are only accessible inside that function. We can create local variables with the same name in different functions. To add a local variable, we can use the following syntax:

local var_name=**<var_value>**

- **Example:**

```
#!/bin/bash
 v1='A'
  v2='B'
my_var () {
local v1='C'
v2='D'
echo "Inside Function"
echo "v1 is $v1."
echo "v2 is $v2."  }
echo "Before Executing the Function"
echo "v1 is $v1."
echo "v2 is $v2."
my_var
echo "After Executing the Function"
```

GRACE COLLEGE

```
echo "v1 is $v1."
echo "v2 is $v2."
```

- **Return Value[Example 1]**

```
#!/bin/bash
print_it () {
    local my_greet="Welcome to Geetanjali."
    echo "$my_greet"
}
my_greet="$(print_it)"
echo $my_greet
```

- **Return Value[Example 2]**

```
#!/bin/bash
#Setting up a return status for a function
print_it () {
    echo Hello $1
   return 5  }
print_it User
print_it Reader
echo The previous function returned a value of $?
```

GRACE COLLEGE